

COMP2113 Programming Technologies

Assignment 2

Deadline: Mar 22, 23:59

Total marks: 100 marks

General Instructions

Read this document carefully before coding.

In this assignment, you need to solve **2 problems**. Your submission will be evaluated by an automatic tester. If your filenames, build process, or output format do not follow this specification, your work may not be graded correctly.

Sample test cases are provided in this document. They are only examples and do not cover all boundary cases.

Input/Output Rules

1. Print outputs to **standard output**.
2. Follow the required output format **exactly** (labels, spaces, line breaks).
3. Extra/missing spaces or lines may cause wrong answers in auto-grading.

Submission

Submit via Moodle VPL.

Put required source files for each problem into one folder (as required by the corresponding VPL activity), and include only required files (`*.cpp`, `*.h`, `Makefile`).

Required filenames

Problem 1

- `1.cpp`

Problem 2

- `gradebook.h`
- `gradebook.cpp`
- `main.cpp`
- `Makefile`

Evaluation

- Auto-grading is based on official test cases.
- Failing output format requirements may lead to zero for a test.
- Sample test cases in this document are for illustration only; additional test cases will be used.

Academic Integrity

Do not copy, share, or plagiarize code. Standard departmental academic integrity policy applies.

Getting Help

Use the course discussion forum for clarification questions. Do not post direct solutions.

Problem 1 [50%]: Pair Differences (k and 0)

Write a C++ program that reads:

- an integer N
- an integer k
- then N integers

and outputs:

1. the number of index pairs (i, j) such that $1 \leq i < j \leq N$ and $|a[i] - a[j]| = k$
2. the number of index pairs (i, j) such that $1 \leq i < j \leq N$ and $a[i] = a[j]$

Input

One line:

```
N k a1 a2 ... aN
```

Constraints

- $1 \leq N \leq 200000$
- $0 \leq k \leq 10000$
- $0 \leq a[i] \leq 10000$

Output

Print **exactly two lines**:

```
DiffK: <number_of_pairs_with_absolute_difference_k>
Diff0: <number_of_pairs_with_equal_values>
```

Compilation and Execution

Compile:

```
g++ -pedantic-errors -std=c++11 -o p1 1.cpp
```

Run with redirected standard input:

```
echo "6 2 1 3 1 5 3 3" | ./p1
```

Notes

1. Use 64-bit integer type (`long long`) for pair counts.
2. If `k = 0`, `DiffK` and `Diff0` should be the same value.

Sample Test Cases

1_1

Input

```
6 2 1 3 1 5 3 3
```

Output

```
DiffK: 9
Diff0: 4
```

1_2

Input

```
8 0 4 4 4 4 4 4 4 4
```

Output

```
DiffK: 28
Diff0: 28
```

Sample Test Case Explanation

For sample `1_1`:

```
N = 6, k = 2
values = [1, 3, 1, 5, 3, 3]
```

Pairs with `|a[i] - a[j]| = 2`:

1. `(1,2)` `(1,5)` `(1,6)`
2. `(2,3)` `(2,4)`
3. `(3,5)` `(3,6)`
4. `(4,5)` `(4,6)`

Total `DiffK = 9`.

Equal-value pairs:

1. Value 1 appears 2 times: $C(2,2) = 1$ pair
2. Value 3 appears 3 times: $C(3,2) = 3$ pairs

Total $Diff0 = 1 + 3 = 4$.

For sample 1_2, all 8 numbers are equal and $k = 0$, so both counts are:

$$C(8,2) = 28$$

Problem 2 [50%]: Gradebook Aggregation (Multiple Files + Makefile)

Write a C++ program that reads grade files from command-line arguments and aggregates students' final scores.

Each line in a grade file should be:

```
<student_id> <score>
```

where:

- `<student_id>`: one word (no spaces)
- `<score>`: integer in `[0, 100]`

A student may appear multiple times across files. The **final score** of a student is the **maximum** valid score found for that student.

Your program must compute:

1. student with highest final score
2. student with lowest final score
3. number of students with final score ≥ 50
4. total number of distinct students

Input (Command-Line Arguments)

Use:

```
int main(int argc, char* argv[])
```

Run format:

```
./gradecounter <file1> <file2> <file3> ...
```

Output

Print summary in exactly 4 lines:

```
Top: <student_id> <final_score>
Bottom: <student_id> <final_score>
Pass: <number_of_students_with_final_score_at_least_50>
Total: <number_of_distinct_students>
```

Error Handling

Handle the following cases:

1. File cannot be opened.
2. A line is invalid (not exactly `<student_id> <score>`, or score not integer, or score out of `[0, 100]`).
3. A file has no valid records.

Required behavior:

1. If (1) or (2) happens for a file, print:

```
<filename>: error processing file
```

2. Print this error message **at most once per file**.
3. Files with no valid records contribute no student data.
4. Empty files are ignored (no valid data to contribute).
5. Print all error lines first (in argument order), then print the 4 summary lines.

Assumptions for this assignment

1. Highest final score is unique.
2. Lowest final score is unique.
3. Across all input files, there is **at least one valid** record.

Separate Compilation Requirement

Use separated compilation:

- declarations in `gradebook.h`
- implementations in `gradebook.cpp`
- program entry in `main.cpp`

Makefile Requirement

Your `Makefile` must include targets:

1. `gradebook.o`: depends on `gradebook.cpp` and `gradebook.h`
2. `main.o`: depends on `main.cpp` and `gradebook.h`
3. `gradecounter`: depends on `gradebook.o` and `main.o`
4. `clean`: removes `gradecounter`, `main.o`, `gradebook.o`

Also:

1. `clean` must be phony (`.PHONY`) so `make clean` always works.
2. Normal build command:

```
make gradecounter
```

3. Clean command:

```
make clean
```

Compilation and Execution

Compile using your Makefile:

```
make clean  
make gradecounter
```

Run with filenames from command line:

```
./gradecounter g1.txt g2.txt
```

If your Makefile is correct, `make clean` should always work, and `make gradecounter` should build the executable.

Sample Test Case

Assume files:

`g1.txt`

```
u1 40  
u2 88  
u3 51  
u2 70  
bad line
```

`g2.txt`

```
u3 49  
u4 100  
u1 60
```

Run:

```
./gradecounter g1.txt g2.txt
```

Output:

```
g1.txt: error processing file
Top: u4 100
Bottom: u3 51
Pass: 4
Total: 4
```

Sample Test Case Explanation

From `g1.txt`:

1. Valid lines: `u1 40`, `u2 88`, `u3 51`, `u2 70`
 2. Invalid line: `bad line`
- Therefore print:

```
g1.txt: error processing file
```

Intermediate final scores after `g1.txt`:

1. `u1 -> 40`
2. `u2 -> max(88, 70) = 88`
3. `u3 -> 51`

From `g2.txt`:

1. `u3 49` does not change `u3` (final remains 51)
2. `u4 100` adds a new student
3. `u1 60` updates `u1` from 40 to 60

Final distinct students:

1. `u1 -> 60`
2. `u2 -> 88`
3. `u3 -> 51`
4. `u4 -> 100`

So:

1. `Top: u4 100`
2. `Bottom: u3 51`
3. `Pass: 4` (all four are `>= 50`)
4. `Total: 4`